

Complete Guide to Neural Network Layers, Parameters, and Data Preprocessing

Neural Network Layers and Their Purposes

1. Dense (Fully Connected) Layers

Purpose: Connect every neuron from one layer to every neuron in the next layer **When to use:**

- Final classification/regression layers
- When you need to learn complex relationships between all features
- Traditional feedforward networks

Parameter Selection:

- **Number of neurons:** Start with powers of 2 (64, 128, 256). More neurons = more capacity but risk of overfitting
- **Activation function:**
 - ReLU for hidden layers (prevents vanishing gradients)
 - Sigmoid for binary classification output
 - Softmax for multi-class classification output
 - Linear for regression output

2. Convolutional Layers (Conv2D)

Purpose: Detect local patterns and features in images **When to use:** Image processing, computer vision tasks

Parameter Selection:

- **Filters:** Start with 32-64 for first layer, double in deeper layers (32→64→128→256)
- **Kernel size:** 3x3 is most common (good balance of detail and efficiency)
- **Stride:** Usually 1 (moves filter one pixel at a time)
- **Padding:** 'same' to maintain image size, 'valid' to reduce size

Theory: Convolution detects edges, textures, and patterns. Multiple filters learn different features.

3. Pooling Layers

Purpose: Reduce spatial dimensions and computational load **When to use:** After convolutional layers to downsample

Types:

- **MaxPooling:** Takes maximum value (preserves strong features)
- **AveragePooling:** Takes average (smoother but may lose important details)
- **Pool size:** Usually 2x2 with stride 2

4. Dropout Layers

Purpose: Prevent overfitting by randomly setting neurons to zero during training **When to use:** When your model is overfitting (training accuracy much higher than validation)

Parameter Selection:

- **Rate:** 0.2-0.5 (20-50% of neurons dropped)
- Place before dense layers, not after output layer

5. Batch Normalization

Purpose: Normalize inputs to each layer, speeds up training and improves stability **When to use:** Between layers in deep networks

Benefits: Allows higher learning rates, reduces internal covariate shift

6. LSTM/GRU Layers

Purpose: Handle sequential data by remembering long-term dependencies **When to use:** Text processing, time series, any sequential data

Parameter Selection:

- **Units:** 64-256 commonly used
- **return_sequences:** True if stacking LSTM layers, False for final layer

Parameter Selection Theory

Learning Rate

- **Too high:** Model jumps around, doesn't converge
- **Too low:** Very slow learning
- **Good starting points:** 0.001 (Adam), 0.01 (SGD)
- **Adaptive:** Use learning rate scheduling to decrease over time

Batch Size

- **Small (16-32):** More noise, may help escape local minima, slower training
- **Large (128-512):** Smoother gradients, faster training, may get stuck
- **Theory:** Larger batches give better gradient estimates but less exploration

Number of Layers (Depth)

- **Shallow (1-3 layers):** Simple problems, limited data
- **Deep (4+ layers):** Complex problems, lots of data
- **Theory:** More layers = more abstract features, but harder to train

Layer Width (Number of Neurons)

- **Narrow:** Less overfitting, may underfit
- **Wide:** More capacity, risk of overfitting
- **Rule of thumb:** Start wide, add regularization; or start narrow and increase if underfitting

Data Scaling - When and Why

When to Scale Data

Always Scale For:

1. **Neural Networks:** Gradient descent works better with normalized inputs
2. **Distance-based algorithms:** KNN, SVM, clustering
3. **Principal Component Analysis (PCA)**
4. **When features have different units** (age vs income vs temperature)

Usually Don't Scale For:

1. **Tree-based models:** Random Forest, Decision Trees (they split on relative values)
2. **Naive Bayes:** Works with probability distributions

Scaling Techniques

1. Min-Max Scaling (Normalization)

$$\text{scaled_value} = (\text{value} - \text{min}) / (\text{max} - \text{min})$$

- **Range:** 0 to 1
- **When to use:** When you know the approximate range of data
- **Pros:** Preserves relationships, bounded output
- **Cons:** Sensitive to outliers

2. Standard Scaling (Z-score normalization)

$$\text{scaled_value} = (\text{value} - \text{mean}) / \text{standard_deviation}$$

- **Range:** Approximately -3 to +3 (unbounded)
- **When to use:** When data follows normal distribution
- **Pros:** Not bounded, handles outliers better
- **Cons:** Output range not fixed

3. Robust Scaling

$$\text{scaled_value} = (\text{value} - \text{median}) / \text{IQR}$$

- **When to use:** When you have many outliers
- **Pros:** Very robust to outliers
- **Uses:** Median and Interquartile Range instead of mean and std

4. Unit Vector Scaling

- **When to use:** When direction matters more than magnitude
- **Common in:** Text processing, recommendation systems

Vectorization Techniques

Text Data Vectorization

1. One-Hot Encoding

How it works: Create binary vector for each unique word **When to use:** Small vocabulary, simple tasks

Pros: Simple, interpretable **Cons:** Huge dimensions, no semantic meaning **Example:** "cat" → [0,0,1,0,0] if cat is 3rd word in vocabulary

2. TF-IDF (Term Frequency-Inverse Document Frequency)

How it works: Weight words by frequency in document vs frequency in corpus **When to use:** Document classification, information retrieval **Pros:** Reduces impact of common words, highlights important terms

Cons: Still sparse, no semantic relationships **Formula:** $\text{TF-IDF} = (\text{word frequency in doc}) \times \log(\text{total docs} / \text{docs containing word})$

3. Word Embeddings

How it works: Dense vectors that capture semantic meaning **Types:**

- **Word2Vec:** Learns from word contexts
- **GloVe:** Global vector representation
- **FastText:** Handles out-of-vocabulary words

When to use: When semantic similarity matters **Pros:** Dense, captures meaning, pre-trained available

Cons: Requires more data to train from scratch

4. Sentence/Document Embeddings

BERT, Sentence-BERT: Transform entire sentences to vectors **When to use:** When sentence-level meaning is important **Pros:** Contextual, state-of-the-art performance **Cons:** Computationally expensive

Categorical Data Vectorization

1. One-Hot Encoding

When to use: Few categories (< 10-20), no ordinal relationship **Example:** Colors [red, blue, green] → [1,0,0], [0,1,0], [0,0,1]

2. Label Encoding

When to use: Ordinal data (small, medium, large) or tree-based models **Example:** Sizes [small, medium, large] → [0, 1, 2]

3. Target Encoding

When to use: High-cardinality categorical features **How it works:** Replace category with average target value for that category **Caution:** Can cause overfitting, use cross-validation

Image Data Preprocessing

1. Normalization

- Divide pixel values by 255 (scale 0-1)
- Or subtract mean and divide by std for each channel

2. Data Augmentation

- **When to use:** Limited image data
- **Techniques:** Rotation, flipping, cropping, brightness changes
- **Purpose:** Increase dataset size and model robustness

Practical Decision Framework

Step 1: Understand Your Data

1. **Data type:** Images, text, tabular, time series?
2. **Data size:** Small (< 1000), medium (1000-100k), large (100k+)?
3. **Problem type:** Classification, regression, clustering?

Step 2: Choose Architecture

- **Tabular data:** Dense layers with dropout
- **Images:** CNN with pooling and dense layers
- **Text:** Embedding + LSTM/GRU or Transformer
- **Time series:** LSTM/GRU or 1D CNN

Step 3: Select Parameters

1. **Start simple:** Few layers, moderate size
2. **Monitor overfitting:** Add dropout/regularization if needed
3. **Scale up gradually:** More layers/neurons if underfitting

Step 4: Preprocessing Pipeline

1. **Scale numerical features:** StandardScaler for normal distributions, MinMaxScaler for bounded data
2. **Encode categoricals:** One-hot for few categories, embeddings for many
3. **Handle text:** TF-IDF for simple tasks, embeddings for complex
4. **Normalize images:** Scale to 0-1, consider augmentation

Common Mistakes to Avoid

1. **Not scaling data** for neural networks
2. **Using wrong activation** in output layer
3. **Too complex model** for simple data
4. **Forgetting validation set** for hyperparameter tuning
5. **Data leakage** in preprocessing (fit scaler on training only)

This framework will help you make informed decisions about layer architecture, parameters, and preprocessing for most machine learning tasks.